

Daily Goals Tracker with GitHub to AWS CodePipeline (CI/CD Project)

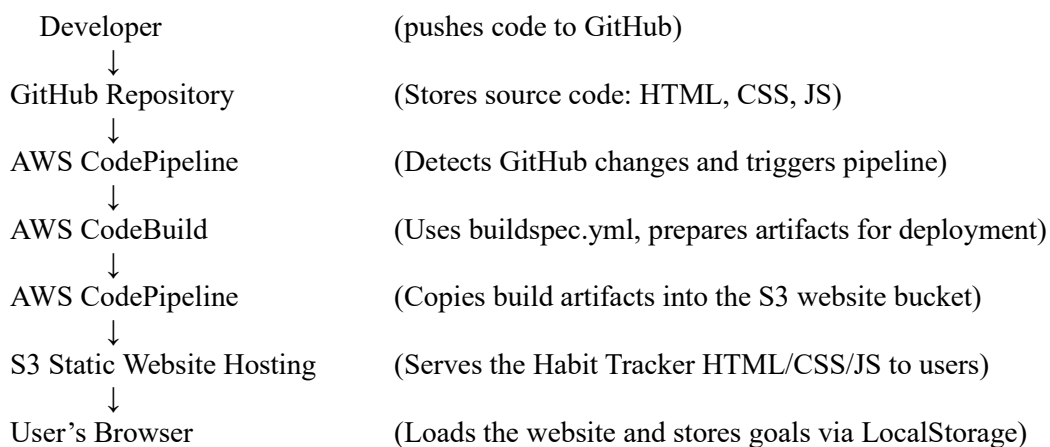
PROJECT DESCRIPTION:

- A tiny browser-based daily goals tracker that stores items in LocalStorage so data persists across refreshes.
- The code is hosted in GitHub and automatically deployed to an AWS S3 website using CodePipeline and CodeBuild.

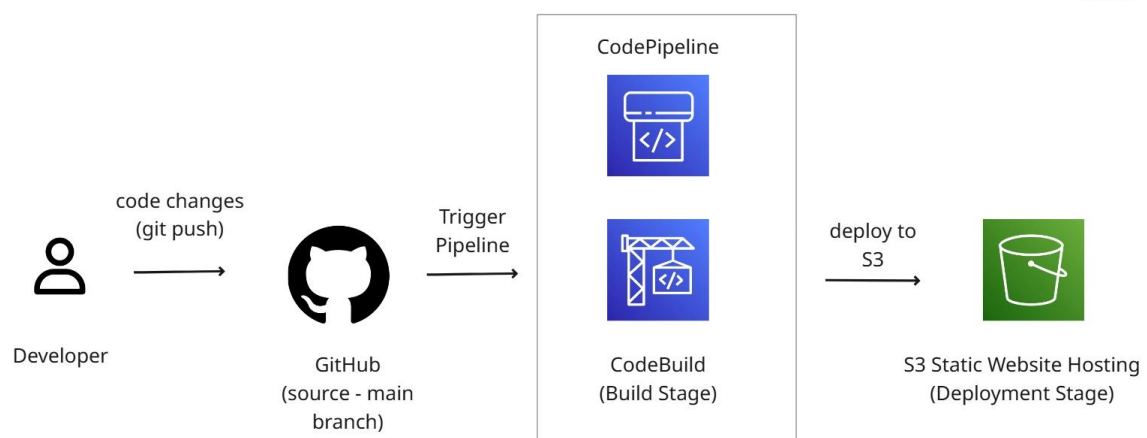
AWS SERVICES USED:

1. Amazon S3 (Static Website Hosting)
2. AWS CodePipeline
3. AWS CodeBuild
4. IAM (Identity & Access Management)

ARCHITECTURAL DIAGRAM:



Architecture visualization



STEP-BY-STEP -- DAILY GOALS HABIT TRACKER WITH AWS CI/CD:

STEP 1 — CREATE GITHUB REPOSITORY:

1. Go to GitHub → New Repository.
2. Repository name: `daily-goals-cicd-pipeline`
3. Description: “Daily Goals Tracker with AWS CI/CD deployment to S3.”
4. Choose **Public**.
5. Click **Create repository**.
6. Upload these project files:

```
`index.html`  
`styles.css`  
`app.js`  
`buildspec.yml`  
`README.md`
```

The screenshot shows a GitHub profile for a user named YOKESH B. The profile includes a circular profile picture of a man, a bio stating he is a Software Developer based in India, and a list of his main tech stack (React, JavaScript, TypeScript, MERN, Tailwind CSS, AWS) and AWS services he knows (EC2, IAM, S3, VPC, Linux, Networking, Lambda). It also mentions his favorite tools (VS Code, GitHub, Vite) and some of his favorite projects (Moments and Tweets). The 'Fun Facts' section lists his favorite anime (Demon Slayer) and hobbies (Anime, movies, and exploring new places). The profile is public and has a repository named 'daily-goals-cicd-pipeline'.

The screenshot shows a GitHub repository page for 'daily-goals-cicd-pipeline' and a terminal window. The repository page displays the repository name, a public status, and a list of files including 'README.md'. The terminal window shows the command prompt and the output of the 'git init' command, indicating that the repository has been initialized.

STEP 2 — CREATE LOCAL APP FILES:

index.html - (Contains UI structure)

```
...
<!doctype html>
<html lang="en">

<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Daily Goals</title>
  <link rel="stylesheet" href="style.css" />
</head>

<body>
  <div class="page-wrapper">
    <header class="site-header">
      <h1 class="title">Daily Goals</h1>
      <p class="subtitle">
        Track what you want to accomplish today.
      </p>
    </header>

    <main class="app">
      <section class="counter">
        <span class="counter-label">Total Goals:</span>
        <span id="goal-count" class="counter-value">0</span>
      </section>

      <form id="goal-form" class="goal-form">
        <input id="goal-input" class="goal-input" placeholder="Enter a goal" maxlength="100" required />
        <button type="submit" class="btn primary">Add</button>
      </form>

      <ul id="goals-list" class="goals-list"></ul>

      <button id="clear-all" class="btn secondary">Clear All</button>
    </main>

    <footer class="footer">
      <p class="footer-text">
        Made with ❤️ by <strong class="footer-author">Yokesh B</strong>
      </p>
    </footer>
  </div>

  <script src="app.js"></script>
</body>

</html>
...
```

app.js - (LocalStorage logic)

```
...
```

```

const form = document.getElementById('goal-form');
const input = document.getElementById('goal-input');
const list = document.getElementById('goals-list');
const clearBtn = document.getElementById('clear-all');

const STORAGE_KEY = 'daily-goals-v1';

function load() {
  const data = JSON.parse(localStorage.getItem(STORAGE_KEY) || '[]');
  list.innerHTML = "";
  data.forEach((g, idx) => {
    const li = document.createElement('li');
    li.textContent = g;
    const del = document.createElement('button');
    del.textContent = 'x';
    del.onclick = () => {
      data.splice(idx,1);
      localStorage.setItem(STORAGE_KEY, JSON.stringify(data));
      load();
    };
    li.appendChild(del);
    list.appendChild(li);
  });
}

form.onsubmit = e => {
  e.preventDefault();
  const val = input.value.trim();
  if (!val) return;
  const data = JSON.parse(localStorage.getItem(STORAGE_KEY) || '[]');
  if (data.length >= 5) {
    alert('Limit 5 goals — delete one first.');
```

```

    return;
  }
  data.push(val);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(data));
  input.value = "";
  load();
};

clearBtn.onclick = () => {
  if (confirm('Clear all goals?')) {
    localStorage.removeItem(STORAGE_KEY);
    load();
  }
};

```

```

window.onload = load;
'''

```

buildspec.yml - (Required for CodeBuild)

```
'''
```

version: 0.2

phases:

install:

commands:

- echo "Nothing to install"

build:

commands:

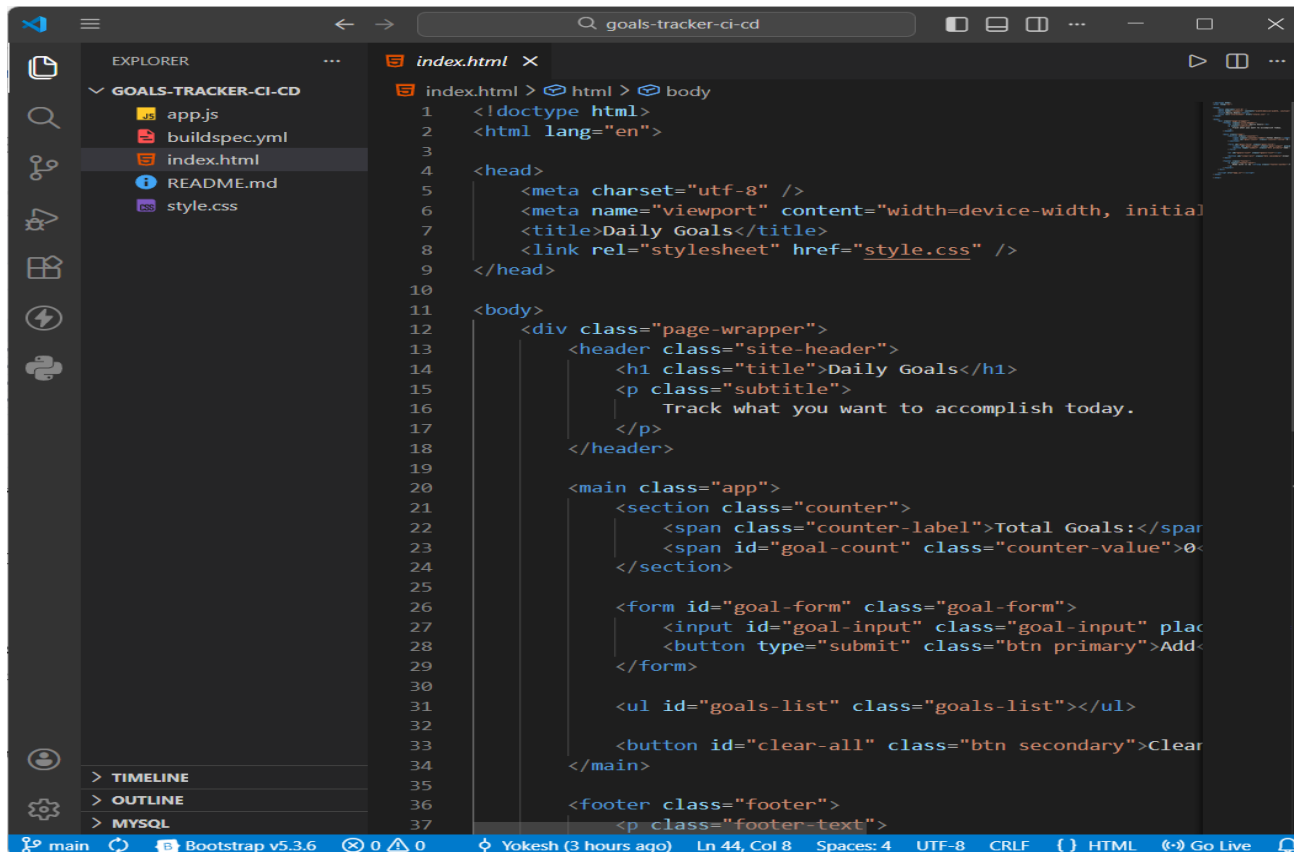
- echo "Static site build complete"

artifacts:

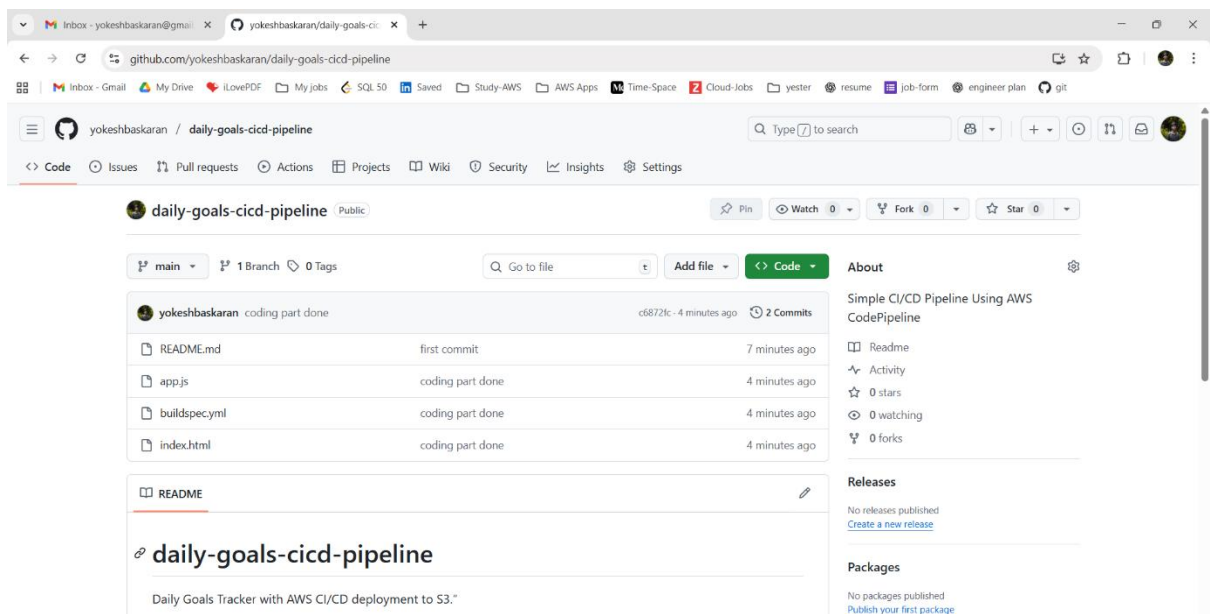
files:

- '/'

...

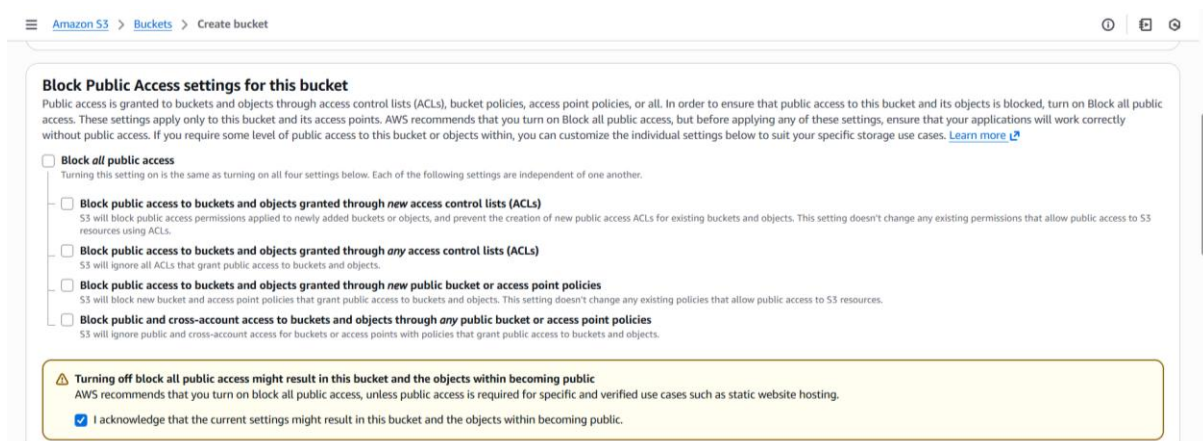
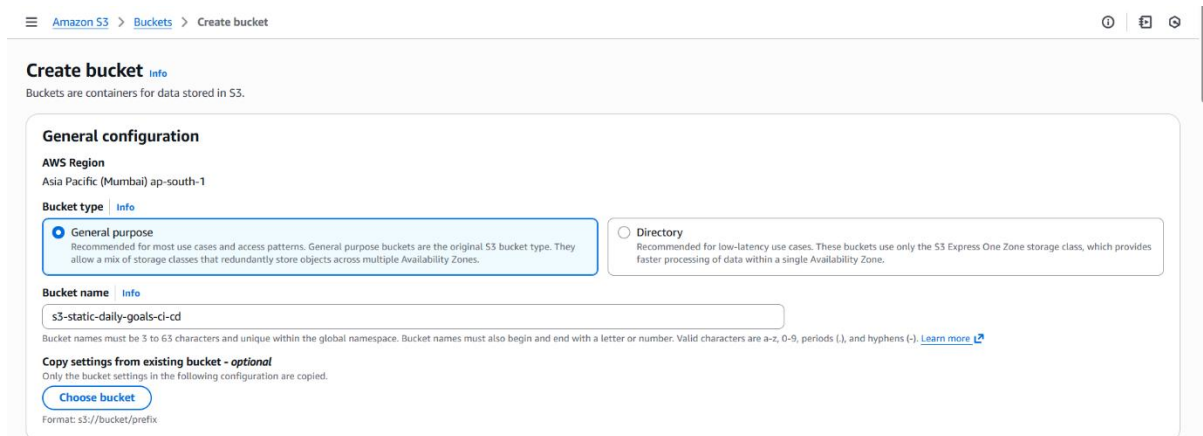
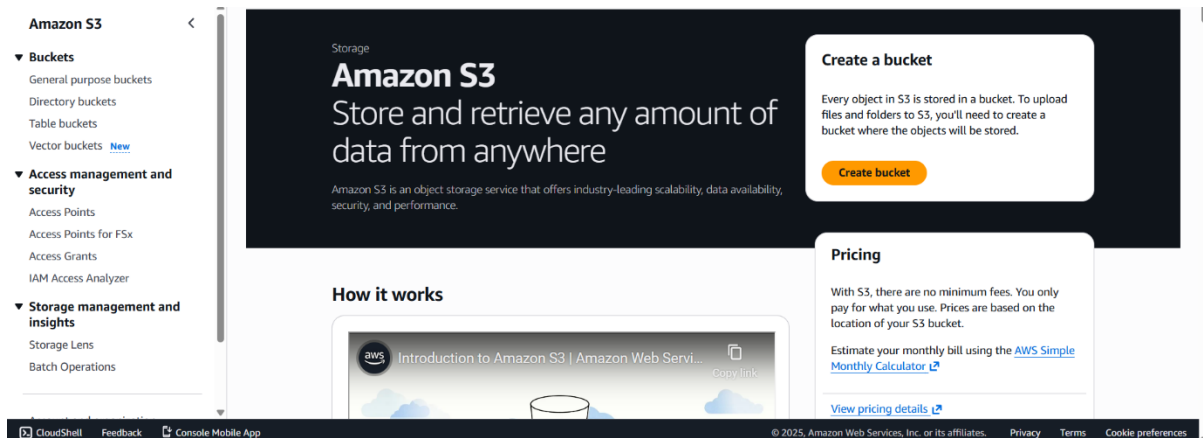


```
1 <!doctype html>
2 <html lang="en">
3
4 <head>
5   <meta charset="utf-8" />
6   <meta name="viewport" content="width=device-width, initial-scale=1">
7   <title>Daily Goals</title>
8   <link rel="stylesheet" href="style.css" />
9 </head>
10
11 <body>
12   <div class="page-wrapper">
13     <header class="site-header">
14       <h1 class="title">Daily Goals</h1>
15       <p class="subtitle">Track what you want to accomplish today.</p>
16     </header>
17
18     <main class="app">
19       <section class="counter">
20         <span class="counter-label">Total Goals:</span>
21         <span id="goal-count" class="counter-value">0</span>
22       </section>
23
24       <form id="goal-form" class="goal-form">
25         <input id="goal-input" class="goal-input" type="text">
26         <button type="submit" class="btn primary">Add Goal</button>
27       </form>
28
29       <ul id="goals-list" class="goals-list"></ul>
30
31       <button id="clear-all" class="btn secondary">Clear All</button>
32     </main>
33
34     <footer class="footer">
35       <p class="footer-text">© 2023 Yokesbaskaran</p>
36     </footer>
37   </div>
38 </body>
39 </html>
```



STEP 3 — CREATE S3 BUCKET FOR WEBSITE HOSTING:

1. Go to AWS Console → S3.
2. Click Create bucket.
3. Name: `daily-goals-<yourname>-site` (s3-static-daily-goals-ci-cd)
4. Region: same region you will use for pipeline.
5. Uncheck “Block all public access.”
6. Create bucket.
7. Open bucket → Properties → scroll to Static website hosting.
8. Enable → Set `index.html`.



STEP 4 — CREATE CODEBUILD PROJECT:

1. AWS Console → CodeBuild → Create build project

2. Name: `daily-goals-code-build`

3. Source provider → GitHub

4. Connect GitHub → choose your repo.

5. Environment:

OS: Amazon Linux 2

Runtime: Standard image

No special runtime needed

6. Buildspec: Use buildspec.yml from repo

7. Artifacts:

Type: Amazon S3

Bucket: choose `daily-goals-<yourname>-site-artifacts`

(Create if needed)

8. Create project.

The screenshot shows the 'Create build project' page in the AWS CodeBuild console. The 'Project configuration' section includes a 'Project name' field with the value 'aws-daily-goals-code-build' and a 'Project type' section with two options: 'Default project' (selected) and 'Runner project'. Below this is an 'Additional configuration' section with a description. The 'Environment' section is expanded, showing 'Provisioning model' with a link to 'Info'. The footer of the console shows 'CloudShell', 'Feedback', 'Console Mobile App', and copyright information for Amazon Web Services, Inc. or its affiliates.

The screenshot shows the 'Environment image' configuration page in the AWS CodeBuild console. It features several sections: 'Environment image' with 'Managed image' (selected) and 'Custom image'; 'Compute' with 'EC2' (selected) and 'Lambda'; 'Running mode' with 'Container' (selected) and 'Instance'; 'Operating system' set to 'Amazon Linux'; 'Runtime(s)' set to 'Standard'; 'Image' set to 'aws/codebuild/amazonlinux-x86_64-standard:5.0'; and 'Image version' set to 'Always use the latest image for this runtime version'. The footer of the console shows 'CloudShell', 'Feedback', 'Console Mobile App', and copyright information for Amazon Web Services, Inc. or its affiliates.

STEP 5 — CREATE CODEPIPELINE (GITHUB → CODEBUILD → S3):

1. AWS Console → CodePipeline

2. Click Create pipeline

3. Name: `daily-goals-pipeline`

4. Allow AWS to create service role

5. Source stage

Provider: GitHub

Repo: `daily-goals-cicd-pipeline`

Branch: `main` or `master`

6. Build stage

Provider: CodeBuild

Project: `daily-goals-build`

7. Deploy stage

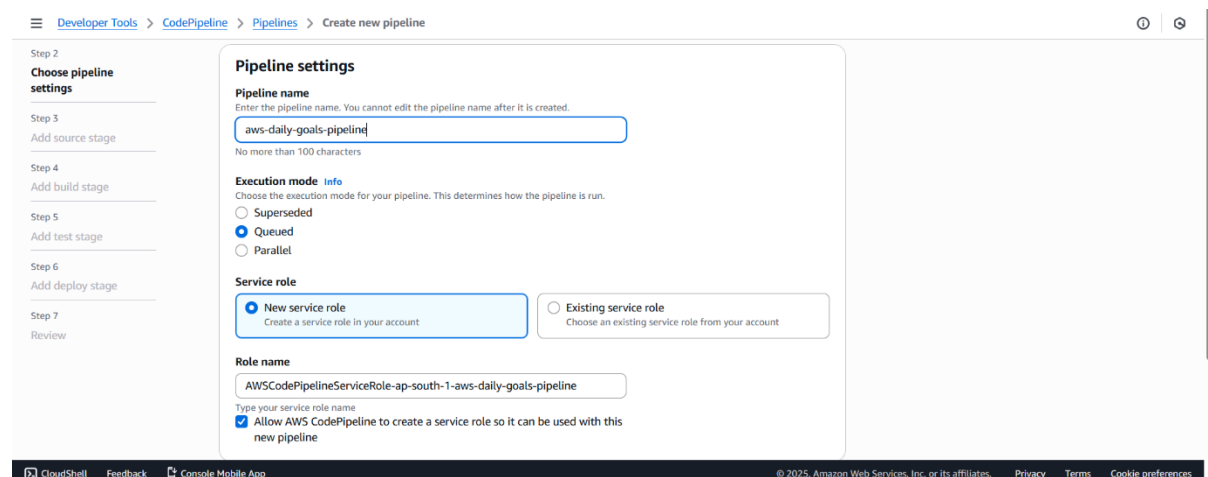
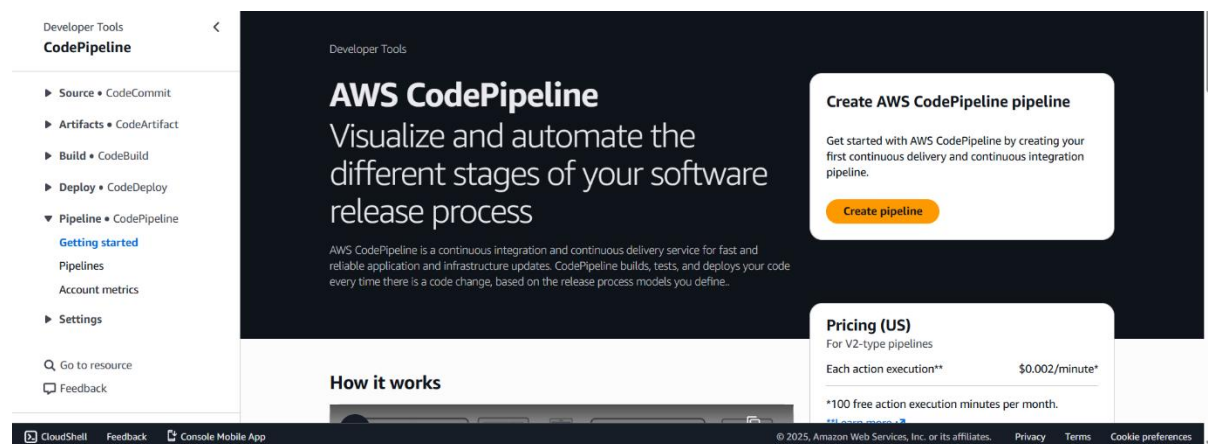
Provider: S3 Deploy

Bucket: `daily-goals-<yourname>-site`

Extract files: yes

8. Click Create pipeline

~ Pipeline now runs automatically.



Developer Tools

CodePipeline

Pipelines

Create new pipeline

Step 1

Choose creation option

Step 2

Choose pipeline settings

Step 3

Add source stage

Step 4

Add build stage

Step 5

Add test stage

Step 6

Add deploy stage

Step 7

Review

Add source stage

Step 3 of 7

Source

Source provider

GitHub (via GitHub App)

Connection

arm:aws:codeconnectionsap- or Connect to GitHub

Repository name

yokeshbaskaran/daily-goals-cicd-pipeline

Default branch

main

Output artifact format

Installing AWS Connector for GitHub - Google Chrome

github.com/apps/aws-connector-for-github/installations/new/permissions?target_id=161866896&target_type=User

Install & Authorize AWS Connector for GitHub

Install & Authorize on your personal account YOKESH B

for these repositories:

All repositories

Only select repositories

Select repositories

Selected 1 repository:

yokeshbaskaran/daily-goals-cicd-pipeline

with these permissions:

Read access to actions, deployments, environments, issues, and metadata

Read and write access to administration, code, commit statuses, pull requests, and repository hooks

Install & Authorize Cancel

Next, you'll be redirected to https://redirect.codestar.aws/return

Developer Tools

CodePipeline

Pipelines

Create new pipeline

Step 1

Choose creation option

Step 2

Choose pipeline settings

Step 3

Add source stage

Step 4

Add build stage

Step 5

Add test stage

Step 6

Add deploy stage

Step 7

Review

Add build stage

Step 4 of 7

Build - optional

Build provider

Commands Other build providers

AWS CodeBuild

Project name

aws-daily-goals-code-build Create project

Define buildspec override - optional

Successfully created aws-daily-goals-code-build in CodeBuild.

Environment variables - optional

Developer Tools

CodePipeline

Pipelines

Create new pipeline

Step 2

Choose pipeline settings

Step 3

Add source stage

Step 4

Add build stage

Step 5

Add test stage

Step 6

Add deploy stage

Step 7

Review

Deploy - optional

Deploy provider

Choose how you want to deploy your application or content. Choose the provider, and then provide the configuration details for that provider.

Amazon S3

Region

Asia Pacific (Mumbai)

Input artifacts

Choose an input artifact for this action. [Learn more](#)

BuildArtifact

Defined by: Build

No more than 100 characters

Bucket

s3-static-daily-goals-ci-cd

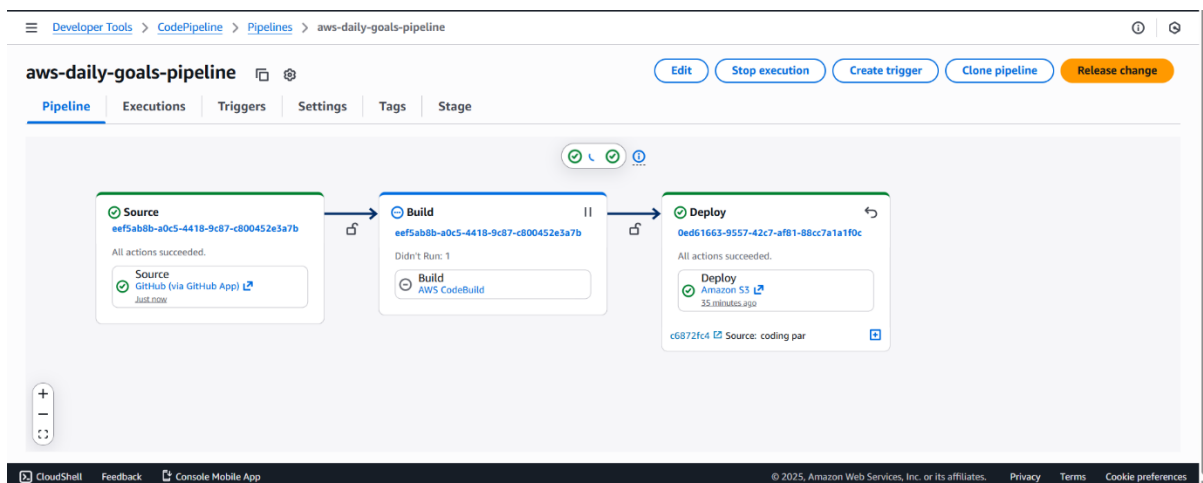
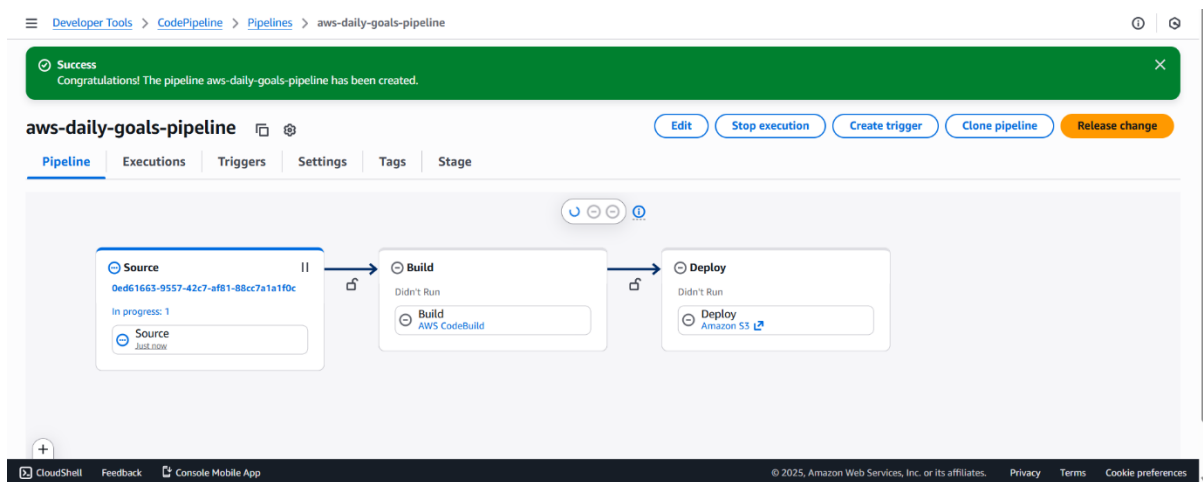
Deployment path - optional

CloudShell

Feedback

Console Mobile App

© 2025, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

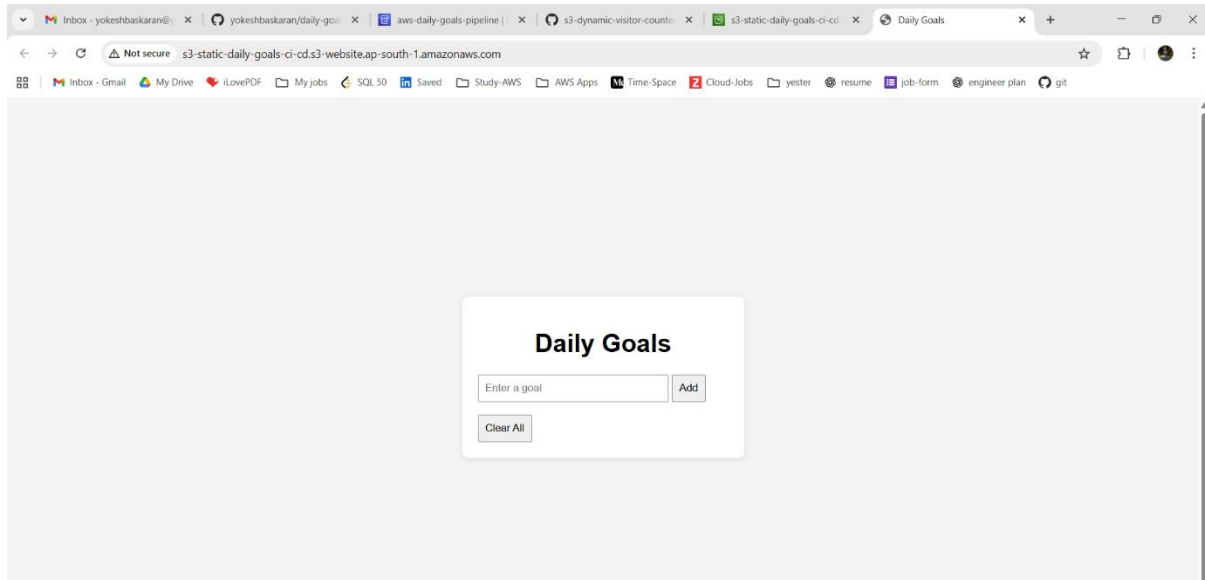


STEP 6 — VIEW LIVE WEBSITE:

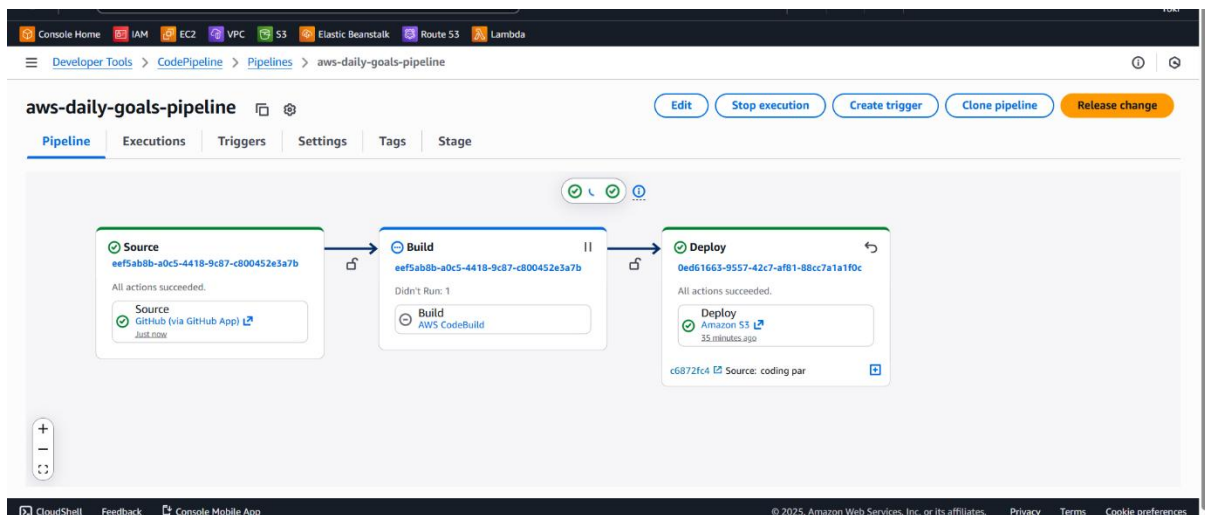
1. Open S3 bucket → Properties.
2. Scroll to Static website hosting.
3. Copy the Website Endpoint URL.

URL: **<http://s3-static-daily-goals-ci-cd.s3-website.ap-south-1.amazonaws.com>**

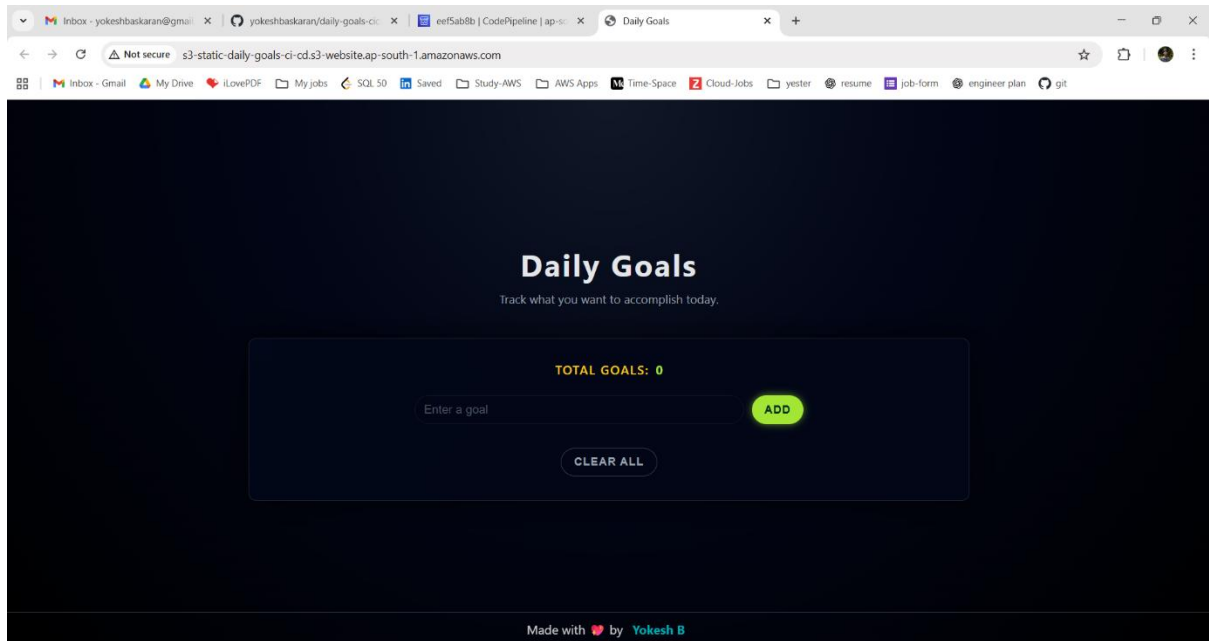
4. Paste in browser.



~ After updating the code in GitHub.



~ The Updated UI in s3 endpoint.



★ HOW THE PIPELINE WORKS (SIMPLE EXPLANATION):

1. You push changes to GitHub.
 2. CodePipeline detects the change automatically.
 3. CodeBuild prepares your static files (even if build is empty).
 4. CodePipeline deploys updated files to S3.
 5. S3 instantly updates your live website.
- This is a full CI/CD pipeline.

PROJECT CONCLUSION:

This project demonstrates the complete lifecycle of a modern web application using GitHub for version control, AWS CodePipeline for automation, CodeBuild for builds, and S3 for static hosting. You created a functional Daily Goals/Habit Tracker that persists data using LocalStorage and learned how to automate deployments using CI/CD. This is a strong portfolio project combining both frontend development and AWS DevOps fundamentals.